OUTPUT:

```
Epoch 1: loss = 1.118, test accuracy = 82.0%
Epoch 2: loss = 0.451, test accuracy = 88.0%
Epoch 3: loss = 0.332, test accuracy = 90.0%


classifying test images
Image  Actual  Predicted  Confidence  Result
    0       7          7       99.9%  correct
    1       2          6       64.1%  wrong
    2       1          1       91.6%  correct
    3       0          0       98.7%  correct
    4       4          4       99.0%  correct
    5       1          1       93.1%  correct
    6       4          4       96.5%  correct
    7       9          9       75.0%  correct
    8       5          2       74.5%  wrong
    9       9          7       76.1%  wrong

Probabilities for test image 0 (actual digit = 7):
  Digit 0:   0.0%
  Digit 1:   0.0%
  Digit 2:   0.0%
  Digit 3:   0.0%
  Digit 4:   0.0%
  Digit 5:   0.0%
  Digit 6:   0.0%
  Digit 7:  99.9%
  Digit 8:   0.0%
  Digit 9:   0.0%
Accuracy per digit (on 200 test images)
Digit 0:  94.1%  (17 images)
Digit 1: 100.0%  (28 images)
Digit 2:  75.0%  (16 images)
Digit 3:  75.0%  (16 images)
Digit 4:  89.3%  (28 images)
Digit 5:  90.0%  (20 images)
Digit 6: 100.0%  (20 images)
Digit 7: 100.0%  (24 images)
Digit 8:  90.0%  (10 images)
Digit 9:  76.2%  (21 images)
Overall accuracy: 90.0%
```

CODE:

```
import numpy as np
np.random.seed(0)

data = np.load("mnist.npz")
x_train, y_train = data["x_train"], data["y_train"]
x_test,  y_test  = data["x_test"],  data["y_test"]
x_train = x_train[:1000] / 255.0
y_train = y_train[:1000]
x_test  = x_test[:200] / 255.0
y_test  = y_test[:200]

NF = 4                                      # number of filters
W1 = np.random.randn(NF, 3, 3) * 0.1        # 4 filters of each 3x3
b1 = np.zeros(NF)
W2 = np.random.randn(NF * 13 * 13, 10) * 0.1  # dense layer of 676 in
b2 = np.zeros(10)

def forward(img):
    conv = np.zeros((NF, 26, 26))
    for f in range(NF):
        for i in range(26):
            for j in range(26):
                patch = img[i:i+3, j:j+3]
                conv[f, i, j] = np.sum(patch * W1[f]) + b1[f]

    relu = np.maximum(0, conv)

    pool = np.zeros((NF, 13, 13))
    for f in range(NF):
        for i in range(13):
            for j in range(13):
                pool[f, i, j] = np.max(relu[f, 2*i:2*i+2, 2*j:2*j+2])

    flat = pool.flatten()
    scores = flat @ W2 + b2

    e = np.exp(scores - np.max(scores))
    probs = e / np.sum(e)
    return conv, relu, pool, flat, probs

def backward(img, label, conv, relu, pool, flat, probs):
    d_scores = probs.copy()
    d_scores[label] -= 1
    dW2 = np.outer(flat, d_scores)
    db2 = d_scores
    d_flat = W2 @ d_scores

    d_pool = d_flat.reshape(NF, 13, 13)

    d_relu = np.zeros((NF, 26, 26))
    for f in range(NF):
        for i in range(13):
            for j in range(13):
                block = relu[f, 2*i:2*i+2, 2*j:2*j+2]
                r, c = np.unravel_index(np.argmax(block), block.shape)
                d_relu[f, 2*i + r, 2*j + c] = d_pool[f, i, j]

    d_conv = d_relu * (conv > 0)

    dW1 = np.zeros_like(W1)
    db1 = np.zeros(NF)
    for f in range(NF):
        for i in range(26):
            for j in range(26):
                dW1[f] += d_conv[f, i, j] * img[i:i+3, j:j+3]
                db1[f] += d_conv[f, i, j]
    return dW1, db1, dW2, db2

lr = 0.01
for epoch in range(3):
    total_loss = 0
    for n in range(len(x_train)):
        img, label = x_train[n], y_train[n]

        conv, relu, pool, flat, probs = forward(img)          # forward
        total_loss += -np.log(probs[label] + 1e-9)            # loss
        dW1, db1, dW2, db2 = backward(img, label, conv, relu, pool, flat, probs)  # backward

        W1 -= lr * dW1;  b1 -= lr * db1                       # update weights
        W2 -= lr * dW2;  b2 -= lr * db2

    correct = sum(np.argmax(forward(x_test[k])[-1]) == y_test[k] for k in range(len(x_test)))
    print(f"Epoch {epoch+1}: loss = {total_loss/len(x_train):.3f}, test accuracy = {correct/len(x_test)*100:.1f}%")

np.set_printoptions(precision=3, suppress=True)

print("classifying test images")
print("Image  Actual  Predicted  Confidence  Result")
for k in range(10):
    probs = forward(x_test[k])[-1]
    pred = np.argmax(probs)
    result = "correct" if pred == y_test[k] else "wrong"
    print(f"{k:5d}  {y_test[k]:6d}  {pred:9d}  {probs[pred]*100:9.1f}%  {result}")

print("Probabilities for test image 0 (actual digit = %d):" % y_test[0])
probs = forward(x_test[0])[-1]
for d in range(10):
    print(f"  Digit {d}: {probs[d]*100:5.1f}%  " + "" * int(probs[d] * 30))

print("Accuracy per digit (on %d test images)" % len(x_test))
preds = np.array([np.argmax(forward(x_test[k])[-1]) for k in range(len(x_test))])
for d in range(10):
    idx = (y_test == d)
    if idx.sum() > 0:
        print(f"Digit {d}: {np.mean(preds[idx] == d)*100:5.1f}%  ({idx.sum()} images)")
print(f"Overall accuracy: {np.mean(preds == y_test)*100:.1f}%")
```